CSE 314 — Operating System Sessional
Online on xv6 Scheduling · Section C2

Longest Weighted Wait

30 minutes. Fresh xv6 checkout, CPUS := 1.

The Idea

Replace the round-robin scheduler with one where waiting builds pressure, and the process with the largest accumulated wait score runs next.

- Every process starts with wait score 0. Before each scheduling decision, add each RUNNABLE process's priority to its score exactly once.
- Pick the RUNNABLE process with the largest resulting score; break ties by process-table order.
- Reset the selected process's score to 0, then run it until it yields, blocks, or exits. Repeat on the next scheduling decision.
- Non-RUNNABLE processes keep their scores unchanged. If none are runnable, wait for work.

Scores count scheduling opportunities, not timer ticks. There is no multi-tick turn or explicit queue. Priority itself never changes automatically.

Specification Parameters

Define the following macros in kernel/param.h:

Macro	Value	Meaning
`MIN_PRIORITY`	1	Lowest priority a process may request.
`MAX_PRIORITY`	10	Highest priority a process may request.
`DEFAULT_PRIORITY`	5	Priority every process starts with.

System Calls

```
int setpriority(int priority);
int getpriority(void);
```

setpriority sets the caller's priority and returns 0. Invalid input resets priority to DEFAULT_PRIORITY and returns −1. It does not change the wait score. getpriority returns the current priority. A fork() child inherits its parent's priority but starts with score 0.

The syscall bodies are supplied in syspriority.c; append them to kernel/sysproc.c. Add the priority and wait-score fields to struct proc and wire the calls through kernel/syscall.h, kernel/syscall.c, user/usys.pl and user/user.h using unused syscall numbers.

Logging

Keep a toggleable log line for every selection, showing the score before resetting it — a global flag or #define that you flip and rebuild is fine:

```
PICK: Process 4 (testloop) chosen with wait 16 at priority 8
```

Test Program

Use the attached testloop.c as-is: save it to user/testloop.c and add $U/_testloop to UPROGS in the Makefile. Calling syntax: testloop <iterations> <priority>.

Sample Session

Three testloop processes, equal work, different priorities, launched one line at a time (scheduler logging off):

```
$ testloop 60 8 &
PID 4: starting 60 iterations at time 38, priority 8
$ testloop 60 4 &
PID 6: starting 60 iterations at time 55, priority 4
$ testloop 60 2 &
PID 8: starting 60 iterations at time 72, priority 2
PID 4: finished at time 92
PID 6: finished at time 132
PID 8: finished at time 155
```

Illustrative output only: PIDs, timestamps and completion order vary. Higher priority builds wait score faster, but the policy does not guarantee CPU shares proportional to priority. Use widely spaced priorities: close values such as 9 and 7 can give identical shares. Exact timings are not part of evaluation.

For a deterministic check, suppose only P1, P2 and P3 are continuously runnable, in that table order, with priorities 8, 4 and 2 and initial scores 0. The first five selections are P1, P1, P2, P1, P3. The second selection is a table-order tie.

Submission

Run these commands from the xv6 checkout. Replace STUDENT_ID with your student ID and submit the patch containing your scheduling changes and test program.

```
git add --all
git diff HEAD > ../STUDENT_ID.patch
```